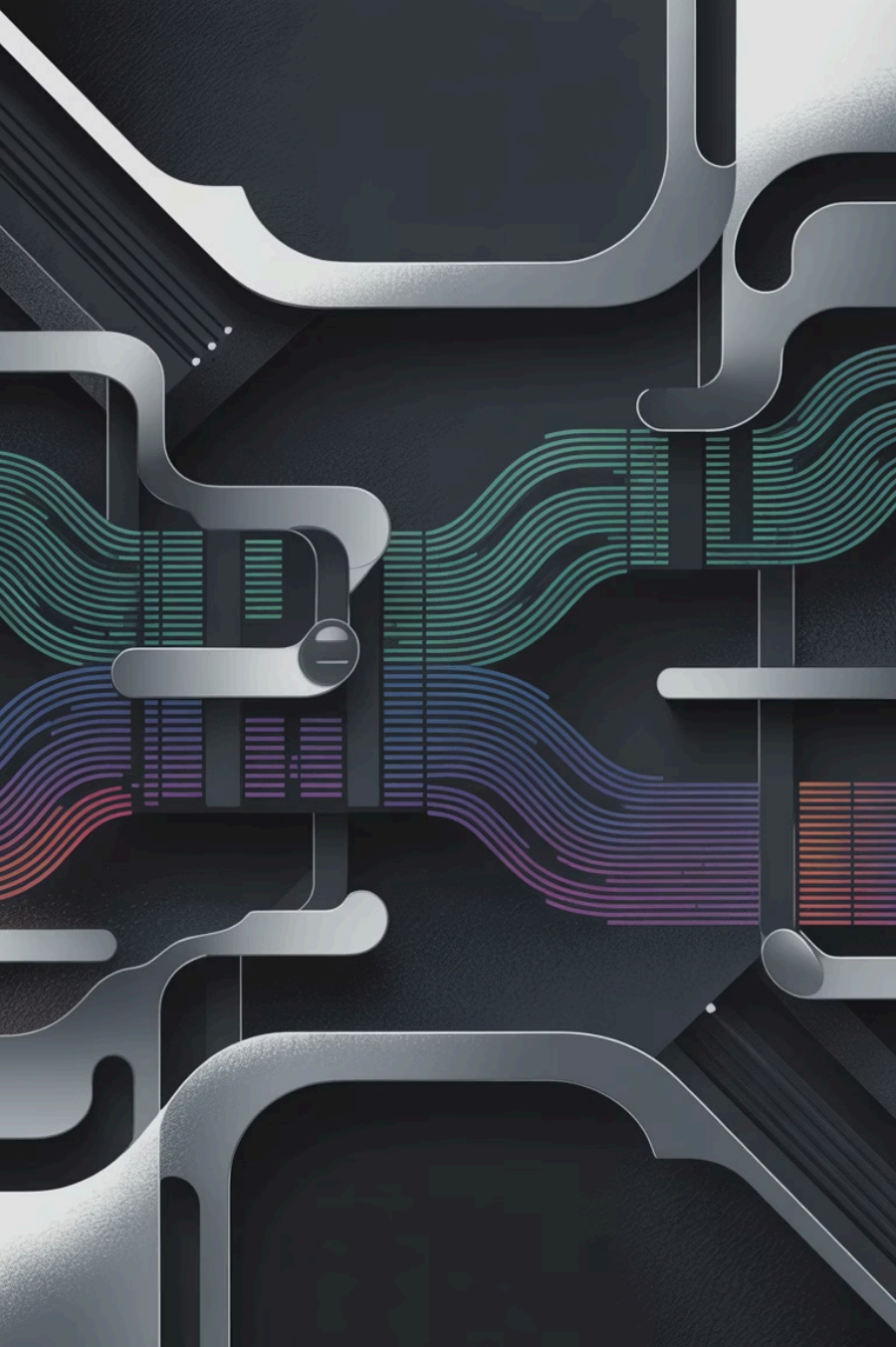




Module 13- Data Engineering for LLM Pipelines

Building the Foundation for AI Engineering Excellence

Today's Session

Deep Dive into RAG Data Preparation

01

Chunking Fundamentals

Understanding semantic boundaries, token limits, and overlap strategies

02

Metadata Engineering

Structuring source attribution, timestamps, and contextual markers

03

Text Preprocessing

Cleaning strategies that preserve semantic meaning for retrieval

Why RAG Data Preparation Matters

The Challenge

LLMs have inherent limitations: outdated training data, no access to proprietary information, and tendency to hallucinate. Raw documents fed directly into vector databases produce poor retrieval results and degrade answer quality.

Effective chunking and preprocessing transform unstructured documents into retrievable knowledge units that maintain semantic coherence while fitting within embedding model constraints.

The Impact

- **Retrieval Precision:** Well-chunked data improves context relevance by 40-60%
- **Answer Quality:** Proper boundaries reduce hallucination and improve factual accuracy
- **System Performance:** Optimized chunks reduce token costs and latency
- **User Trust:** Rich metadata enables source attribution and transparency



The RAG Architecture Context

RAG systems operate in two phases: **indexing** (offline data preparation) and **retrieval** (runtime query processing). Data preparation happens during indexing, where documents are chunked, embedded, and stored in vector databases.

Your preprocessing decisions directly impact retrieval quality. Chunk size affects semantic completeness, overlap preserves context across boundaries, and metadata enables filtering and attribution. These choices cascade through the entire system architecture.

Core Chunking Strategies



Character-Based

Split by fixed character count (e.g., 1000 chars). Simple but breaks semantic units. Use when speed matters more than quality.



Sentence-Based

Preserve sentence boundaries using NLP parsers. Maintains grammatical coherence but variable chunk sizes.



Structural

Split by document structure (headings, paragraphs, sections). Best semantic preservation for structured docs.



Token-Based

Split by embedding model token limits (e.g., 512 tokens). Ensures compatibility but may break semantics.

Most production systems use **hybrid approaches** that combine structural splitting with token limits and sentence awareness.



Chunk Size Trade-offs

Small Chunks (128-256 tokens)

- Higher retrieval precision
- Less noise in context
- Risk losing coherence
- More chunks to search

Medium Chunks (256-512 tokens)

- Balanced approach
- Good for most use cases
- Maintains context
- Standard practice

Large Chunks (512-1024 tokens)

- Maximum context
- Fewer chunks to manage
- May include irrelevant info
- Higher token costs

Rule of thumb: Start with 512 tokens (roughly 400 words) and adjust based on document structure and retrieval metrics.

Understanding Overlap

1

No Overlap

Clean boundaries but context may be lost at chunk edges. Critical information spanning boundaries becomes fragmented.

2

With Overlap

Redundant content but preserves context. Information at boundaries appears in multiple chunks, improving retrieval.

3

Smart Overlap

Semantic-aware overlap that preserves complete thoughts. Best retrieval quality at the cost of complexity.

Standard practice: **10-20% overlap** (50-100 tokens for 512-token chunks). Balance redundancy against retrieval completeness based on your use case.

Chunking by Sentences

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

# .from_tiktoken_encoder makes chunk_size count TOKENS (not characters)
text_splitter = RecursiveCharacterTextSplitter.from_tiktoken_encoder(
    encoding_name="cl100k_base", # tokenizer for text-embedding-3 / GPT-4 family
    chunk_size=512,             # now genuinely 512 tokens
    chunk_overlap=50,           # 50 tokens
    separators=["\n\n", "\n", ". ", " ", ""],
)

chunks = text_splitter.split_text(document_text)
```

The `RecursiveCharacterTextSplitter` tries separators in order: paragraphs first, then sentences, then words. This preserves semantic units while respecting size constraints.

The `separators` list defines the hierarchy of split points. It attempts to split on double newlines (paragraphs) first, falling back to single newlines, then sentence boundaries (periods), and finally spaces if needed.

Chunking by Document Structure

```
from langchain_text_splitters import
MarkdownHeaderTextSplitter

headers_to_split_on = [
    ("#", "Header 1"),
    ("##", "Header 2"),
    ("###", "Header 3"),
]

markdown_splitter = MarkdownHeaderTextSplitter(
    headers_to_split_on=headers_to_split_on
)

md_header_splits =
markdown_splitter.split_text(markdown_document)
```

Structural Splitting Benefits

- Preserves document hierarchy in metadata
- Each chunk knows its section context
- Enables hierarchical retrieval strategies
- Natural semantic boundaries

Particularly effective for technical documentation, research papers, and structured knowledge bases where headings denote topic shifts.

Token-Aware Chunking

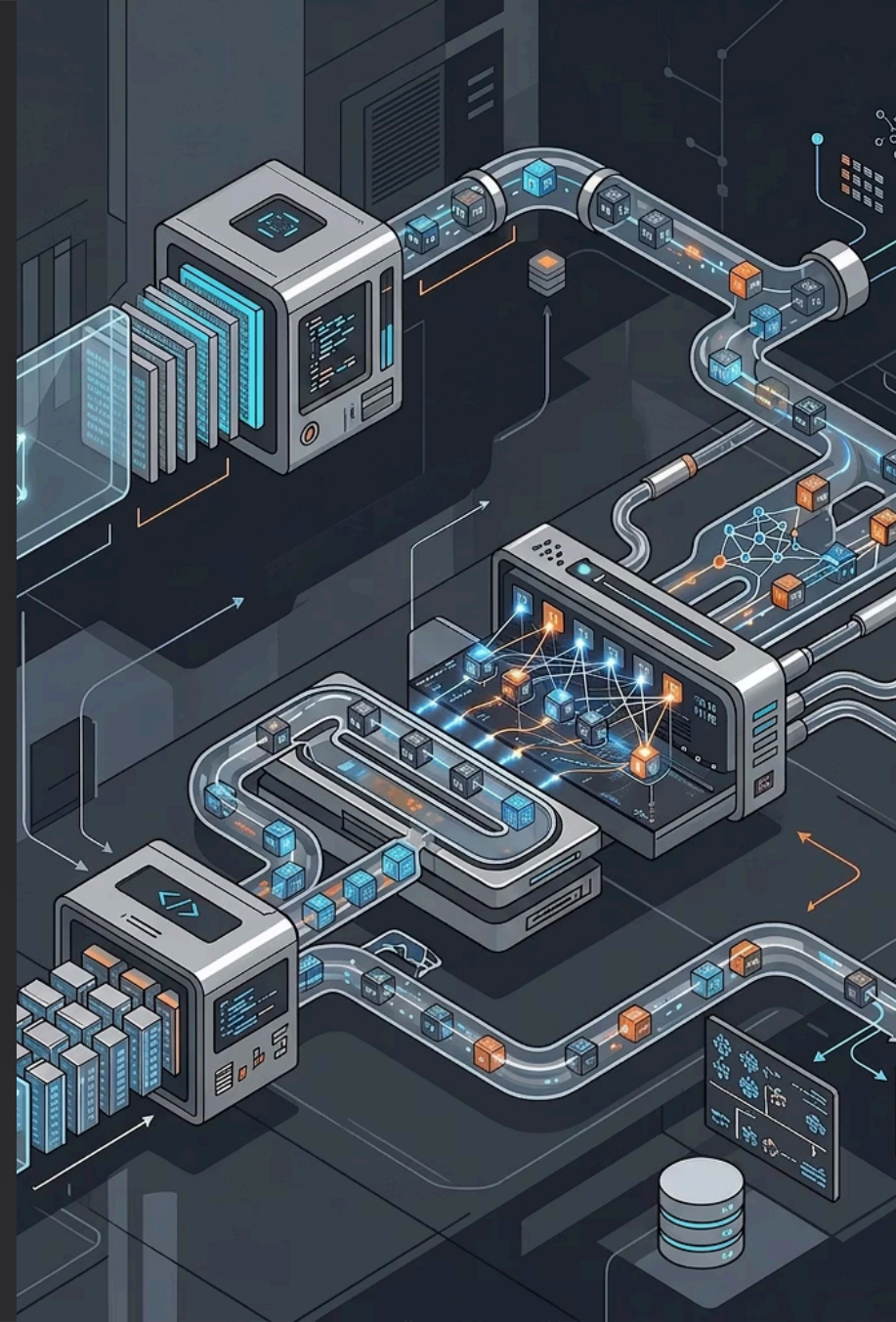
Embedding models have hard token limits (e.g., 512 for many models, 8191 for OpenAI text-embedding-3). Exceeding these limits causes truncation or errors. Token-aware chunking ensures compatibility.

```
from langchain_text_splitters import TokenTextSplitter

token_splitter = TokenTextSplitter(
    chunk_size=512,          # tokens
    chunk_overlap=50,        # tokens
    encoding_name="cl100k_base", # OpenAI's tokenizer (text-embedding-3 /
                                # GPT-4)
)

token_chunks = token_splitter.split_text(text)
```

Critical insight: Tokens $\neq$ words. English text averages 1.3 tokens per word. Always measure in tokens, not characters or words, when working with LLM pipelines.



Metadata Engineering Principles

Source Attribution

Track document source, filename, URL, or database record ID.
Enables citation and provenance tracking.

Temporal Context

Timestamp creation/update dates. Critical for time-sensitive information and enabling recency filtering.

Structural Context

Section headers, page numbers, hierarchy position. Helps LLM understand the document structure and relationships.

Content Categorization

Document type, topic tags, classification labels. Enables filtered retrieval and improves relevance.

Practical Metadata Structure

```
{  
  "chunk_id": "doc_123_chunk_5",  
  "source": "https://docs.example.com/api-guide.html",  
  "source_type": "documentation",  
  "title": "API Authentication Guide",  
  "section": "OAuth 2.0 Implementation",  
  "subsection": "Token Refresh Flow",  
  "page_number": 23,  
  "created_at": "2024-01-15T10:30:00Z",  
  "updated_at": "2024-03-20T14:45:00Z",  
  "version": "2.1",  
  "author": "platform-team",  
  "tags": ["authentication", "oauth", "security"],  
  "chunk_index": 5,  
  "total_chunks": 47,  
  "token_count": 487  
}
```

Store metadata alongside embeddings in your vector database. Most systems support JSON metadata fields that enable filtering during retrieval queries.

Metadata-Driven Retrieval

Basic Similarity Search

```
results = vectorstore.similarity_search(  
    query="How do I refresh OAuth tokens?",  
    k=5  
)
```

Returns the 5 most similar chunks by embedding distance alone. No filtering or prioritization.

Metadata-Filtered Search

```
results = vectorstore.similarity_search(  
    query="How do I refresh OAuth tokens?",  
    k=5,  
    filter={  
        "source_type": "documentation",  
        "tags": {"$in": ["authentication", "oauth"]},  
        "updated_at": {"$gte": "2024-01-01"}  
    }  
)
```

Searches only recent, relevant documentation chunks. Dramatically improves precision.

Text Preprocessing: What to Clean

Remove

- Excessive whitespace and tabs
- HTML tags and markup artifacts
- Navigation elements and boilerplate
- Advertisements and sidebars

Preserve

- Punctuation (semantic meaning)
- Code blocks and formatting
- Lists and bullet structures
- Technical notation and symbols

Normalize

- Unicode encoding (UTF-8)
- Line endings (consistent \n)
- Quote marks and apostrophes
- Whitespace to single spaces

Key principle: Preserve semantic meaning. Don't over-clean. Punctuation, capitalization, and formatting often carry important information for retrieval and generation.

TEXT...

—フ—フ— ||リ



Placeholder text represented by horizontal bars of varying lengths, simulating a document layout.

When to Keep Punctuation

Meaningful Punctuation

- **Question marks:** Signal inquiry vs statement
- **Exclamation points:** Indicate emphasis or urgency
- **Colons/semicolons:** Show relationships between clauses
- **Quotes:** Distinguish direct speech or technical terms
- **Hyphens:** Compound words and technical terminology

Code and Technical Content

Never strip punctuation from code blocks, API endpoints, configuration files, or technical notation. These characters are semantically critical:

- `user.authenticate()` vs `user authenticate`
- `GET /api/v2/users` vs `GET api v2 users`

Your preprocessing pipeline must distinguish between prose and technical content.

Basic Cleaning Pipeline

```
import re
import unicodedata

def clean_text(text: str) -> str:
    """Basic cleaning while preserving semantic meaning (UTF-8 native)."""
    # Canonicalize Unicode WITHOUT ASCII-folding (keeps accents, symbols, non-Latin)
    text = unicodedata.normalize("NFC", text)

    # Normalize curly quotes/apostrophes to straight ASCII (on the real codepoints)
    text = text.replace("\u201C", "").replace("\u201D", "") # " " -> "
    text = text.replace("\u2018", "").replace("\u2019", "") # ' ' -> '

    # Collapse runs of spaces/tabs to a single space, but PRESERVE newlines
    text = re.sub(r"[ \t]+", " ", text)

    # Remove control characters, keeping \t \n \r
    text = re.sub(r"[\x00-\x08\x0B\x0C\x0E-\x1F\x7F]", "", text)

    # Trim trailing spaces on each line, then overall
    text = "\n".join(line.strip() for line in text.split("\n"))
    return text.strip()
```

This preserves punctuation, maintains readability, and removes only problematic characters.

Handling Special Document Types



PDFs

Use `pypdf` or `pdfplumber` for extraction. Handle multi-column layouts, headers/footers, and embedded tables separately. Watch for encoding issues.



Code Files

Parse with language-specific tools. Split by functions/classes, preserve docstrings as metadata, and maintain syntax highlighting markers.



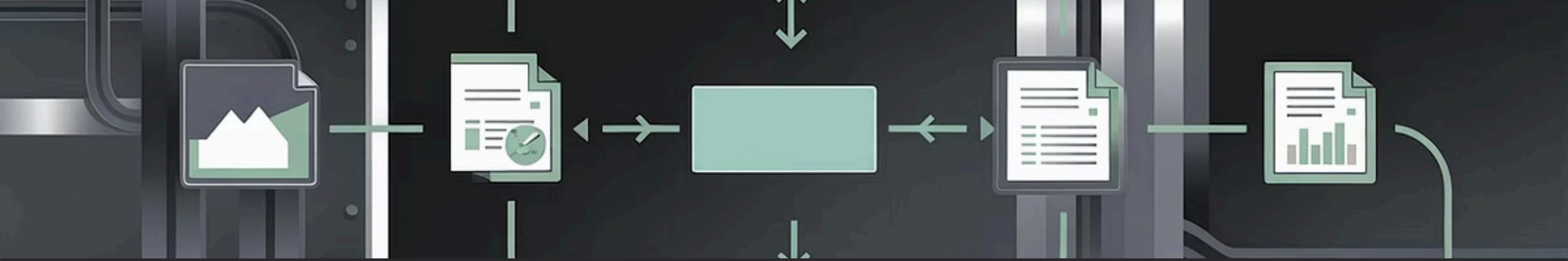
Markdown

Use structural splitters that preserve headers as metadata. Convert to plain text or HTML depending on downstream needs.



Web Content

Use BeautifulSoup or Trafilatura to extract main content. Remove navigation, ads, and scripts. Preserve links and images as metadata.



Complete RAG Preprocessing Pipeline

Pipeline Stage 1: Document Loading

```
from langchain_community.document_loaders import (  
    PyPDFLoader,  
    UnstructuredMarkdownLoader,  
    WebBaseLoader,  
)  
  
pdf_loader = PyPDFLoader("research_paper.pdf")  
md_loader = UnstructuredMarkdownLoader("docs/guide.md")  
web_loader = WebBaseLoader("https://example.com/article")  
  
documents = []  
documents.extend(pdf_loader.load())  
documents.extend(md_loader.load())  
documents.extend(web_loader.load())
```

LangChain loaders return `Document` objects with `page_content` (text) and `metadata` (dict). Each loader automatically populates basic metadata like `source`.

Pipeline Stage 2: Metadata Enrichment

```
from datetime import datetime, timezone

def enrich_metadata(documents: list) -> list:
    """Add custom metadata to each document."""
    for i, doc in enumerate(documents):
        # tz-aware UTC timestamp -> ISO-8601 with offset, sorts chronologically
        doc.metadata["processed_at"] = datetime.now(timezone.utc).isoformat()
        doc.metadata["doc_index"] = i

        source = doc.metadata.get("source", "")
        if source.endswith(".pdf"):
            doc.metadata["source_type"] = "pdf"
        elif source.endswith(".md"):
            doc.metadata["source_type"] = "markdown"
        elif source.startswith("http"):
            doc.metadata["source_type"] = "web"

    return documents

documents = enrich_metadata(documents)
```

Pipeline Stage 3: Text Cleaning

```
def preprocess_documents(documents: list) -> list:
    """Clean document text while preserving meaning"""
    for doc in documents:
        text = doc.page_content

        # Remove excessive whitespace
        text = re.sub(r'\s+', ' ', text)

        # Remove page numbers and headers (if pattern is clear)
        text = re.sub(r'Page \d+', '', text)

        # Remove URLs if not needed (or extract to metadata)
        urls = re.findall(r'https?:\/\/[^\s]+', text)
        if urls:
            doc.metadata['referenced_urls'] = urls
            text = re.sub(r'https?:\/\/[^\s]+', '[URL]', text)

        doc.page_content = text.strip()

    return documents

documents = preprocess_documents(documents)
```

Pipeline Stage 4: Chunking with Overlap

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

# Token-based sizing so chunk_size means TOKENS, consistent with the embedder's limit
text_splitter = RecursiveCharacterTextSplitter.from_tiktoken_encoder(
    encoding_name="cl100k_base",
    chunk_size=512,
    chunk_overlap=50,
    separators=["\n\n", "\n", ". ", " ", ""],
)

# split_documents propagates each source doc's metadata onto its chunks
chunks = text_splitter.split_documents(documents)

for i, chunk in enumerate(chunks):
    src = chunk.metadata.get("source", "doc")
    chunk.metadata["chunk_id"] = f"{src}_chunk_{i}" # stable-ish, avoids cross-run collisions
    chunk.metadata["chunk_index"] = i
    chunk.metadata["total_chunks"] = len(chunks)

print(f"Created {len(chunks)} chunks from {len(documents)} documents")
```

Pipeline Stage 5: Embedding Generation

```
from langchain_openai import OpenAIEmbeddings
from langchain_chroma import Chroma

# OPENAI_API_KEY must be set in the environment (never hard-code)
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
# Note: text-embedding-3-small is 1536-dim by default.
# Only pass dimensions=... if you deliberately want FEWER dims (Matryoshka), e.g. 512.

vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    persist_directory="./chroma_db",
    collection_name="rag_knowledge_base",
)

print(f"Indexed {len(chunks)} chunks with embeddings")
```

The vector store handles embedding generation automatically and stores vectors alongside metadata for efficient retrieval.

Complete Pipeline: Putting It All Together

```
def build_rag_pipeline(sources: list, output_dir: str):
    """Complete RAG preprocessing pipeline"""

    # 1. Load documents
    documents = load_documents(sources)
    print(f"Loaded {len(documents)} documents")

    # 2. Enrich metadata
    documents = enrich_metadata(documents)

    # 3. Clean text
    documents = preprocess_documents(documents)

    # 4. Chunk with overlap
    chunks = chunk_documents(documents)
    print(f"Created {len(chunks)} chunks")

    # 5. Generate embeddings and store
    vectorstore = create_vectorstore(chunks, output_dir)

    return vectorstore

# Execute pipeline
vectorstore = build_rag_pipeline(
    sources=['docs/', 'papers/', 'https://example.com'],
    output_dir='./vector_db'
)
```


Validation and Quality Checks

1

Chunk Size Distribution

Plot token counts to verify most chunks are near target size. Watch for outliers that may indicate splitting issues.

2

Metadata Completeness

Verify all required fields are populated. Check for null values or missing source attribution.

3

Content Quality

Sample random chunks and manually review. Ensure semantic coherence and proper boundary handling.

4

Retrieval Testing

Run test queries and evaluate top-k results. Measure relevance using human evaluation or automated metrics.

Common Pitfalls and Solutions

Pitfall: Over-Cleaning

Removing punctuation, lowercasing everything, or stripping too much formatting destroys semantic meaning. Your retrieval suffers because embeddings can't distinguish nuance.

Solution: Clean only problematic characters. Preserve punctuation, capitalization, and structure that carries meaning.

Pitfall: Ignoring Document Structure

Fixed-size character splitting breaks semantic units mid-sentence or mid-paragraph.

Solution: Use recursive or structural splitters that respect document hierarchy.

Pitfall: Insufficient Metadata

Storing only embeddings without source, timestamp, or context makes it impossible to filter or cite sources.

Solution: Capture rich metadata at load time and enrich through pipeline stages.

Pitfall: No Overlap

Information at chunk boundaries is fragmented and may not be retrieved effectively.

Solution: Use 10-20% overlap to preserve context across boundaries.

Optimization Strategies



Batch Processing

Process documents in parallel using multiprocessing or async operations. Embedding APIs support batching for better throughput.



Caching

Cache embeddings and preprocessing results. Recompute only when source documents change.



Incremental Updates

Design pipelines for incremental processing. Only process new or modified documents, not entire corpus.

Measuring Pipeline Success

85%

Retrieval Precision

Percentage of retrieved chunks
that are relevant to the query

92%

Answer Accuracy

Percentage of generated answers
that are factually correct

2.3s

Average Latency

Mean time from query to
response, including retrieval and
generation

40%

Cost Reduction

Token savings from optimized
chunking vs. naive approaches

Track these metrics over time as you refine your preprocessing pipeline. A/B test different chunking strategies and measure impact on downstream quality.

Next Steps: Advanced Techniques

Hybrid Search

Combine dense embeddings with sparse keyword search (BM25) for improved retrieval. Especially effective for technical terms and proper nouns.

Semantic Chunking

Use LLMs to identify natural semantic boundaries instead of fixed-size splits. More computationally expensive but better quality.

Multi-Vector Retrieval

Generate multiple embeddings per chunk at different granularities. Retrieve at coarse level, then re-rank with fine-grained embeddings.

Contextual Compression

Use an LLM to extract only relevant sentences from retrieved chunks before passing to generation. Reduces noise and token costs.



Thanks